

Projeto MecaniQA

Autor: <Gustavo Prates Caetano>
<Matheus Costa Rodrigues>
<Nallanda Novato>
<Thainá Araujo Salgado>
<Vinicius Martins Lebrão>
<Vitor Hugo Lima Silva Bittencourt >

Vitória da Conquista, 2026

Agenda

O objetivo dessa apresentação é apresentar o desenvolvimento e validação da API RESTful criada para gerenciar peças e serviços mecânicos, cumprindo os requisitos da OAT 1.

1. Item:

Contextualização do problema.

2. Item:

Orientação a Objetos no Projeto.

3. Item:

Arquitetura e Padrão Singleton.

4. Item:

Controllers e API REST

5. Item:

Considerações Finais

Contextualização do problema



O Problema da Oficina:

Alta dificuldade na gestão manual de estoque de peças e no tabelamento de serviços prestados, gerando perda de tempo e erros de orçamento.

Objetivo da Fase 1:

Desenvolver o *Back-end* fundamental do sistema. Uma API RESTful capaz de centralizar o cadastro, listagem, atualização e exclusão (operações CRUD) de Peças e Serviços.

Infraestrutura Atual:

Nesta etapa, focamos na validação das regras de negócio. O armazenamento de dados opera puramente **em memória RAM** do servidor (Listas Java), dispensando banco de dados externo, como pedido na OAT.

Orientação a objetos no Projeto **unex**

Definição das Entidades

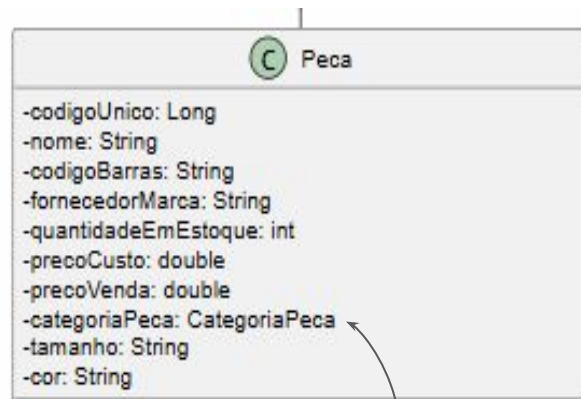
- Criação das classes principais **Peca** e **Servico** representando o domínio real da oficina.

Tipagem Rigorosa (Tipos Primitivos e Wrappers)

- Uso de **Long** para chaves/IDs, **String** para descrições e nomes, e **double/int** para valores monetários e estoque.

Associação de Domínio

- Inclusão do enum **CategoriaPeca** para padronizar e restringir os tipos de peças cadastradas (motor, freios, suspensão, etc.).



Orientação a objetos no Projeto

Modificadores de Acesso(Encapsulamento)

O encapsulamento protege os dados utilizando atributos privados (-) e métodos públicos (+) como Getters e Setters para acesso controlado.

```
+ Peca()  
+ getCodigoUnico(): Long  
+ setCodigoUnico(codigoUnico: Long)  
+ getNome(): String  
+ setNome(nome: String)
```

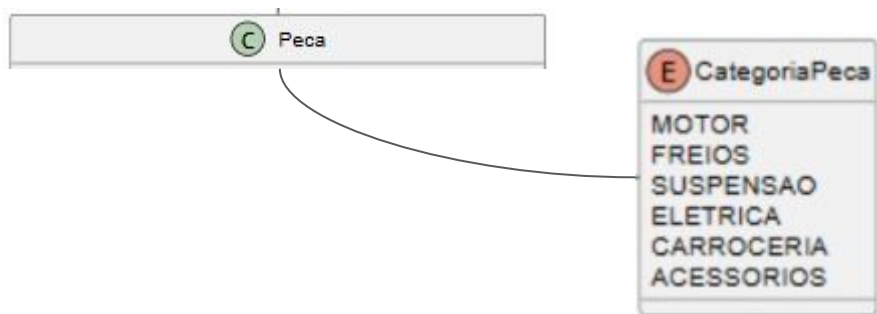
```
public Long getCodigoUnico() { 1 usage  👤 Gustavo  
    return codigoUnico;  
}  
public void setCodigoUnico(Long codigoUnico) {  
    this.codigoUnico = codigoUnico;  
}  
public String getNome() { no usages  👤 Gustavo Pra  
    return nome;  
}  
public void setNome(String nome) { no usages  👤  
    this.nome = nome;  
}
```

Orientação a objetos no Projeto **unex**

Enums

Um **Enum** é usado para definir um conjunto fixo de valores, facilitando a padronização dos dados. No exemplo, o `CategoriaPeca` define as categorias possíveis de uma peça, como **MOTOR, FREIOS, SUSPENSAO e ELETRICA**.

No diagrama, o **Enum CategoriaPeca** está associado à classe **Peca**, indicando que cada peça pode possuir uma categoria definida por esses valores. Isso evita valores diferentes para representar a mesma categoria e mantém o sistema mais organizado.

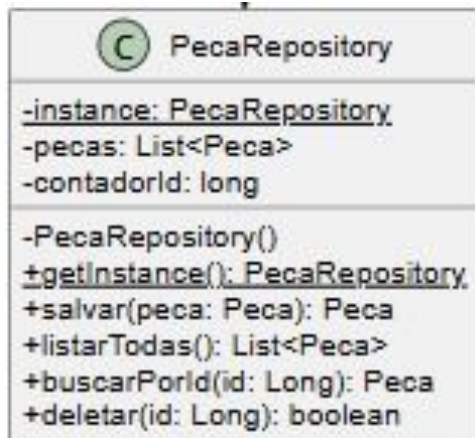


```
public enum CategoriaPeca {  
    MOTOR, no usages  
    SUSPENSAO, no usages  
    FREIOS, no usages  
    ELETRICA, no usages  
    ACESSORIOS no usages  
}
```

Arquitetura e Padrão Singleton

O que é o padrão Singleton?

O **Singleton** garante uma única instância da classe, permitindo seu acesso por diferentes partes da aplicação. No projeto, ele é utilizado nos **Repositories** para centralizar o armazenamento e acesso aos dados em memória.



Arquitetura e Padrão Singleton

Como o Singleton é implementado?

`private PecaRepository()` → impede a criação com `new`

`private static PecaRepository instance` → armazena a única instância.

`getInstance()` → cria a instância caso ela não exista e depois reutiliza a mesma.

`List<Peca>` → mantém as peças armazenadas em memória.

```
public class PecaRepository { 5 usages  @ Gustavo Pra
    private static PecaRepository instance; 3 usages
    private List<Peca> pecas; 5 usages
    private long contadorId; 2 usages
    private PecaRepository() { 1 usage  @ Gustavo Pra
        this.pecas = new ArrayList<>();
        this.contadorId = 1;
    }
    public static PecaRepository getInstance() {
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}
```

Arquitetura e Padrão Singleton

Como o Singleton é implementado?

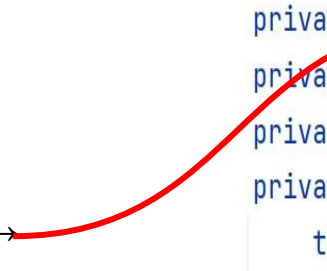
`private PecaRepository()` → impede a criação com `new`

`private static PecaRepository instance` → armazena a única instância.

`getInstance()` → cria a instância caso ela não exista e depois reutiliza a mesma.

`List<Peca>` → mantém as peças armazenadas em memória.

```
public class PecaRepository { 5 usages  @ Gustavo Pra
    private static PecaRepository instance; 3 usages
    private List<Peca> pecas; 5 usages
    private long contadorId; 2 usages
    private PecaRepository() { 1 usage  @ Gustavo Pra
        this.pecas = new ArrayList<>();
        this.contadorId = 1;
    }
    public static PecaRepository getInstance() {
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}
```



Arquitetura e Padrão Singleton

Como o Singleton é implementado?

`private PecaRepository()` → impede a criação com `new`

`private static PecaRepository instance` → armazena a única instância.

`getInstance()` → cria a instância caso ela não exista e depois reutiliza a mesma.

`List<Peca>` → mantém as peças armazenadas em memória.

```
public class PecaRepository { 5 usages  @ Gustavo Pra
    private static PecaRepository instance; 3 usages
    private List<Peca> pecas; 5 usages
    private long contadorId; 2 usages
    private PecaRepository() { 1 usage  @ Gustavo Pra
        this.pecas = new ArrayList<>();
        this.contadorId = 1;
    }
    public static PecaRepository getInstance() {
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}
```

Arquitetura e Padrão Singleton

Como o Singleton é implementado?

`private PecaRepository()` → impede a criação com `new`

`private static PecaRepository instance` → armazena a única instância.

`getInstance()` → cria a instância caso ela não exista e depois reutiliza a mesma.

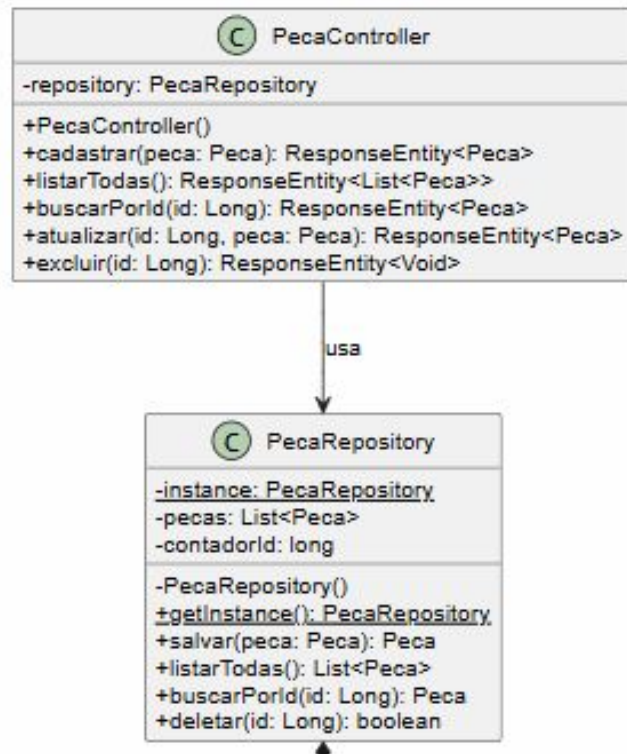
`List<Peca>` → mantém as peças armazenadas em memória.

```
public class PecaRepository { 5 usages  @ Gustavo Pra
    private static PecaRepository instance; 3 usages
    private List<Peca> pecas; 5 usages
    private long contadorId; 2 usages
    private PecaRepository() { 1 usage  @ Gustavo Pra
        this.pecas = new ArrayList<>();
        this.contadorId = 1;
    }
    public static PecaRepository getInstance() {
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}
```

Arquitetura e Padrão Singleton

Funcionamento na Arquitetura

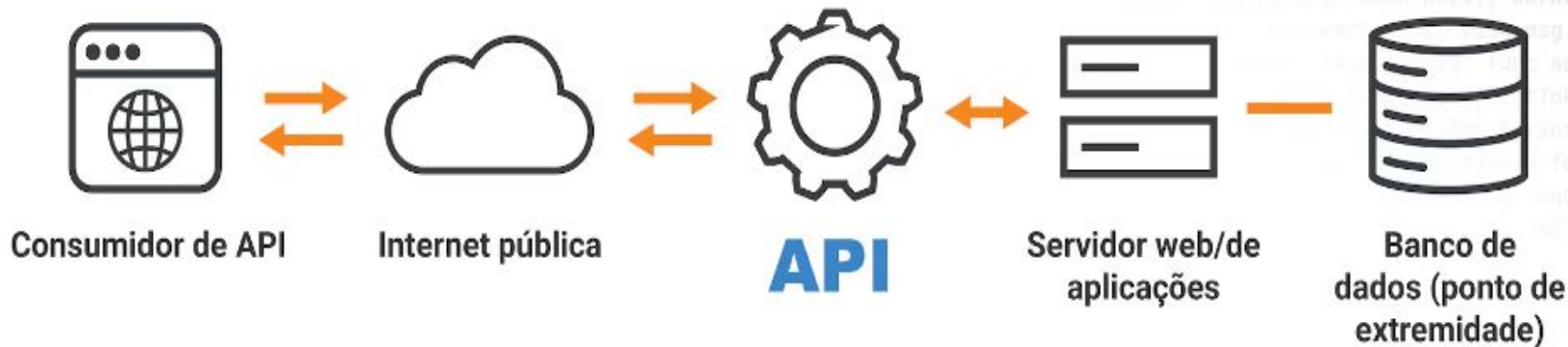
- Os **Controllers** utilizam os Repositories.
- Os **Repositories** armazenam os objetos em List.
- PecaRepository possui uma única instância de PecaRepository.
- ServicoRepository segue o mesmo padrão.
- Os dados permanecem disponíveis enquanto a aplicação estiver em execução.



Controllers e API Rest

API REST

A **API REST** permite a comunicação entre clientes e servidor por meio de **requisições HTTP**, utilizando endpoints (rotas) específicos para cada recurso.



Verbos HTTP

GET = Consulta dados @GetMapping

POST = Cria um novo recurso @PostMapping

PUT = Atualiza um recurso completo @PutMapping

PATCH = Atualiza parte de um recurso

DELETE = Remove um recurso @DeleteMapping

Controllers e API Rest



Os **Controllers** recebem e tratam as requisições HTTP, direcionando cada operação para o **Repository** e retornando uma resposta adequada ao cliente por meio dos **Status HTTP**.

Cliente → Controller → Repository → Controller → Cliente

200 OK — Sucesso

201 Created — Recurso criado

404 Not Found — Recurso não encontrado

400 Bad Request — Requisição inválida

Controllers e API Rest



- O **Controller** recebe a requisição HTTP do cliente via `@PostMapping`.
- Os dados são processados e salvos pelo **Repository**.
- O sistema retorna o status **201 Created** junto com o recurso cadastrado encapsulado em um `ResponseEntity`.

201 Created — Recurso criado

`@PostMapping`   Gustavo Prates Caetano

```
public ResponseEntity<Servico> cadastrar(@RequestBody Servico servico) {  
    Servico servicoSalvo = repository.salvar(servico);  
    return ResponseEntity.status(HttpStatus.CREATED).body(servicoSalvo); // 201 Created  
}
```

Rotas da API/Postman



Post: `http://localhost:8081/api/pecas`

Get: `http://localhost:8081/api/pecas`

Put: `http://localhost:8081/api/pecas/Indice`

Delete: `http://localhost:8081/api/pecas/Indice`

Considerações Finais



- **Arquitetura Implementada:** O Back-end foi construído aplicando fortemente os pilares de Orientação a Objetos.
- **Padrões de Projeto:** O uso do padrão Singleton permitiu a simulação eficaz de um banco de dados em memória.
- **Validação de Rotas:** A API RESTful demonstrou resiliência nos testes realizados via Postman, retornando os Status HTTP corretos.